Features

În ordinea și cu numerotarea din `PLAN.pdf` · bifezi când merge · Sebi

Starea la `f3bd97b`, rulată: verde = merge · cărămiziu = scris, dar nu merge · gri = de scris. F1–F5 le faci singur, cu ce știi; singura noutate din ele e Flyway, în F1. De la F6, câte o unealtă nouă pe feature.

1 Contul meu

vizitator · utilizator · administrator

Nou: Flyway (V1) — lecția `spring-flyway` înainte (ex. 1–3 făcute, 4–8 de terminat)

- ☐ **F1.1** Aplicația compilează și CI-ul e verde. — compilare
  `BatteryController.java:43`, `CommandExecutionMapper.java:14, :16`, `BatteryCommandServiceIT.java:99`
- ☐ **F1.2** Schema o face Flyway; Hibernate doar o verifică (`validate`). — `V1__schema.sql`
  `V1_schema.sql` (spațiu, un singur `_`) e ignorat · `ddl-auto: update` · `flyway.enabled: false`
- ☐ **F1.3** Vizitatorul își face cont de proprietar: nume, prenume, vârstă, email, parolă. — `register`
  oricine își poate alege `userType: ADMIN` (`AuthServiceImpl.java:55-56`)
- ☐ **F1.4** Utilizatorul se autentifică și primește un token. — `login`
  merge
- ☐ **F1.5** Utilizatorul autentificat își vede contul și permisiunile. — `getCurrentUser`
  de scris · `/auth/**` e public azi
- ☐ **F1.6** Adminul din `.env` poate administra. — la pornire
  se creează, dar fără permisiuni (`permissions: []`)
- ☐ **F1.7** Administratorul creează un cont `OPERATOR`, `USER` sau `AGENT`. — `addUser`
  403 pentru toată lumea (`hasAuthority('ADMIN')`); nu salvează parola, vârsta, tipul
- ☐ **F1.8** Erorile au corp complet, iar o cerere greșită e 400, nu 401. — `GlobalExceptionHandler`
  corp gol la 404 / 403 / 409 · email invalid → 401 (`/error`)
- ☐ **F1.9** Testele pe endpoint-uri protejate pornesc de la login real. — teste
  3 teste de login merg (H2), dar niciunul nu refolosește tokenul · 5 teste cu `@WithMockUser`

Reguli

- Email valid și unic (409); parolă de minim 8 caractere; vârstă ≥ 18. Prin înregistrare se face doar `USER`; `ADMIN` vine din `.env`.
- Login greșit → 401, același mesaj pentru email inexistent și parolă greșită. Parola nu apare în niciun răspuns.
- Permisiunile vin din tip, dintr-o singură hartă: fiecare șir din `hasAuthority(...)` există în `UserPermissions`.

2 Casa cu bateria ei

administrator · operator · proprietar

Cu ce știi: da (a doua migrare, `V2` = exercițiul 5 din `spring-flyway`)

- ☐ **F2.1** Administratorul adaugă o casă: nume, putere PV, limite de import și export, proprietar. Bateria se creează odată cu ea. — `createHouse`
  403 (`'house:manage'`) · bateria nu se leagă de casă · randament 0.95 în loc de 95.0
- ☐ **F2.2** Administratorul modifică o casă (`PUT`, `PATCH`) sau o scoate din simulare (`enabled`). — `updateHouse` · `patchHouse`
  403 · `enabled` lipsește din cerere
- ☐ **F2.3** Administratorul șterge o casă fără istoric. — `deleteHouse`
  403 · cu istoric dă 500, trebuie 409
- ☐ **F2.4** Proprietarul își vede casa; administratorul și operatorul văd toate casele. — `getAllHouses` · `getHouseById`
  403 · lista dă toate casele oricui · operatorul primește 403 pe casă
- ☐ **F2.5** Proprietarul își vede bateria. — `getBattery`
  merge pentru proprietar și admin · operatorul 403
- ☐ **F2.6** Casa altcuiva → 403 cu corp complet; casă inexistentă → 404 cu corp complet. — erori
  străinul primește 500 · 404 cu corp gol

Reguli

- Nume obligatoriu și unic (400 / 409); puterile > 0; proprietar inexistent → 404. Baterie implicită: 10 kWh, SoC 50%, 3 kW / 3 kW, randament 95%.
- Casa cu istoric nu se șterge (409); o scoți din simulare cu `enabled: false`.

3 Comand bateria — poarta

proprietar · operator · administrator

Cu ce știi: da

- [ ] **F3.1** Proprietarul schimbă puterile maxime de încărcare și de descărcare. `updateConfiguration`
  `updateConfiguration` întoarce `null` (corpul s-a șters)
- [ ] **F3.2** Proprietarul trimite o comandă: `CHARGE`, `DISCHARGE` sau `IDLE`, cu puterea în kW. `sendCommand`
  `executeCommand` întoarce `null` · cererea are `targetSoc`, nu `powerKw`
- [ ] **F3.3** Proprietarul vede istoricul comenzilor casei, cu rezultatul fiecăreia. `getCommands`
  `getCommandsForHouse` întoarce lista goală
- [ ] **F3.4** Comanda peste limite se salvează `REJECTED`, cu motivul. `executeCommand`
  validatorul există (4 teste verzi); lipsește legătura
- [ ] **F3.5** Testele verifică SoC-ul de după și statusul, nu doar codul HTTP. teste
  1 test cu `@Disabled("F3")` de scos · 2 teste roșii în `BatteryCommandServiceIT` (rulat pe nume) · aserțiune greșită la `testActualizareConfig`

Reguli

- O comandă acționează un tick (o oră virtuală). Peste puterea maximă sau cu SoC-ul ieșit din 10–90% → `REJECTED` (`BATTERY_POWER_LIMIT` / `BATTERY_SOC_LIMIT`), iar bateria nu se schimbă. Orice altă eroare nu e `REJECTED`.
- `CHARGE` 2 kW pe SoC 50, 10 kWh, 95% → `SUCCESS`, SoC 69.

4 O zi simulată

administrator · proprietar · operator

Cu ce știi: da (tick la cerere, log simplu)

- [ ] **F4.1** La prima pornire cu profilul `demo` există 10 case cu baterii și 10 proprietari. la pornire
  de scris
- [ ] **F4.2** Administratorul rulează un tick: fiecare casă activă primește PV, consum, preț și un rând de telemetrie. `runSimulationTick`
  de scris · `EnergyEngine` e scris și testat, dar nu-l cheamă nimeni
- [ ] **F4.3** Proprietarul vede ultimele 50 de măsurători ale casei, cele mai noi primele. `getHouseTelemetry`
  de scris · `TelemetryRepository` e comentat
- [ ] **F4.4** Operatorul vede starea comunității: PV total, consum total, SoC mediu. `getCommunityState`
  de scris

Reguli

- Aceeași sămânță, aceeași casă, aceeași oră → aceleași valori. Noaptea PV = 0; la 12:00 PV = vârful casei.
- Un tick = o oră virtuală, de la 2026-06-01T00:00. Tick doar `ADMIN` (403); telemetria casei altcuiva → 403. A doua pornire nu mai creează nimic.

5 Banii casei

proprietar · operator

Cu ce știi: da (`BigDecimal` e Java de bază)

- [ ] **F5.1** La fiecare tick, importul casei se înregistrează ca `BUY`, exportul ca `SELL`, la prețul tick-ului. în tick
  `recordTransaction` n-are niciun apelant · lucrează cu `double`
- [ ] **F5.2** Proprietarul vede tranzacțiile casei lui. `getHouseFinancialSummary`
  cale fără `/api/v1` · fără verificare de proprietar
- [ ] **F5.3** Operatorul vede tranzacțiile întregii comunități. `getCommunityFinancialSummary`
  `community` stă pe locul unui `{id}`, fără `/api/v1` · testul trece doar cu `@WithMockUser`

Reguli

- Valoarea = kWh × preț, pe `BigDecimal`, 2 zecimale `HALF_UP`. Prețul vine din telemetria aceluiași tick. Cel mult un rând pe casă și tick.
- Proprietarul vede doar casa lui (403). Cele mai noi primele.

6 Testele rulează pe MySQL real

tu · CI

Nou: Testcontainers + failsafe — lecția `spring-testcontainers` înainte (gata)

- ☐ **F6.1** Testele de integrare rulează pe MySQL real, la `mvn -B verify`; H2 iese. — `mvn -B verify`
  `pom.xml` n-are failsafe; clasele `*IT` nu rulează la `mvn test` (B9); H2 în pom
- ☐ **F6.2** Toate clasele cu `@SpringBootTest` devin `*IT extends AbstractIT`, inclusiv `SolarSyncBrokerApiApplicationTests` și `AdminBootstrapTests`. — `AbstractIT`
  de făcut
- ☐ **F6.3** Migrările V1–V5 sunt verificate în teste (`SchemaIT`). — `SchemaIT`
  de scris
- ☐ **F6.4** CI pe `pull_request`, cu check obligatoriu; un PR cu test roșu nu intră în `master`. — `ci.yaml`
  de făcut

7 Logarea zilei simulate

administrator

Nou: logare structurată (MDC) — lecția `spring-logare-structurata` înainte (de făcut)

- ☐ **F7.1** Fiecare linie de log din timpul unui tick poartă `tickId`, iar cele per casă și `houseId`. — MDC
  de scris
- ☐ **F7.2** Pe EC2 logurile ies ca JSON și un tick se filtrează după `tickId`. — profilul `ec2`
  de scris

8 Starea rețelei

operator · administrator

Nou: Micrometer + Prometheus + Grafana — lecția `devops/metrici-prometheus-grafana` înainte (de făcut)

- ☐ **F8.1** La fiecare tick se calculează încărcarea transformatorului comun și se salvează starea. — în tick
  de scris · `GridProperties` există, nefolosit
- ☐ **F8.2** Operatorul vede starea rețelei după ultimul tick. — `getGridState`
  de scris
- ☐ **F8.3** Comunitatea se vede live în Grafana. — `/actuator/prometheus`
  de scris

Reguli

- Transformator 50 kW, marjă 10% → limită 45 kW. Import și export separat: ≤ 45 `OK`, până la 50 `WARNING`, peste `CRITICAL`, cu kW-ii în plus.

9 Comunitatea văzută de un agent

orice client MCP

Nou: server MCP — lecția `java/spring-ai/mcp-server` înainte (de făcut)

- ☐ **F9.1** Un client MCP se conectează cu un token și vede cele 8 tool-uri de citire. — server MCP
  de scris
- ☐ **F9.2** Fiecare tool dă exact cât vede utilizatorul tokenului, ca prin REST. — tool-uri
  de scris

10 Propun → aprob → execut

agent · operator

Nou: tool-uri MCP de scriere — lecția `mcp-server`, partea a doua (de făcut)

- ☐ **F10.1** Agentul propune un plan: o decizie per casă, trecută prin validator fără să atingă bateria. — `proposePlan`
  de scris · `BatteryCommandValidator` există
- ☐ **F10.2** Operatorul vede rulările și deciziile fiecăreia, valide și respinse, și aprobă o rulare: deciziile valide devin comenzi reale. — `getAgentRuns` · `approveAgentRun`
  de scris
- ☐ **F10.3** `AgentMode` hotărăște ce se întâmplă cu planul: respins, doar salvat, așteaptă aprobare, sau se execută imediat, tot prin poartă. — moduri
  de scris · enum-ul există

11 Brokerul pe reguli — primul microserviciu

agent

Nou: client MCP, a doua aplicație — lecția `java/spring-ai/mcp-client` înainte (de făcut)

- ☐ **F11.1** Aplicația separată `solar-sync-broker-agent` citește comunitatea prin MCP și propune un plan la interval. — aplicație nouă
  de scris
- ☐ **F11.2** Planner pe reguli (prag 24 h, 0,75 / 1,4); rețeaua `WARNING` / `CRITICAL` are prioritate. — `RuleBasedPlanner`
  de scris
- ☐ **F11.3** Agentul oprit → API-ul merge mai departe. — —
  de verificat

12 Brokerul cu AI

agent

Nou: Spring AI `ChatClient` — lecția `java/spring-ai/chat-client` înainte (de făcut)

- ☐ **F12.1** Un al doilea planner, cu model AI; alegi din configurare care rulează. — `LlmPlanner`
  de scris
- ☐ **F12.2** Fără cheie API sau la eroare, agentul trece pe reguli și scrie un `WARN`. — fallback
  de scris
- ☐ **F12.3** Comparația reguli vs. AI pe aceeași zi simulată. — tabel
  de scris

13 Tick-ul anunță, ceilalți reacționează

sistem

Nou: Kafka — lecția `java/spring/spring-kafka` înainte (de făcut)

- ☐ **F13.1** După commit, tick-ul publică evenimente; banii și rețeaua devin ascultători. — `telemetry.recorded` · `grid.state`
  de scris
- ☐ **F13.2** Același eveniment primit de două ori nu dublează nimic. — idempotență
  de scris
- ☐ **F13.3** Kafka UI arată topicurile și mesajele (profilul `messaging`). — Ops
  de scris

14 Simulatorul pleacă din monolit

sistem

Nou: topicurile ca singur contract — lecția `java/microservicii/al-doilea-serviciu` înainte (de făcut)

- ☐ **F14.1** `solar-sync-broker-simulator` produce măsurătorile și le publică. — aplicație nouă
  de scris
- ☐ **F14.2** API-ul le consumă și merge și fără simulator, doar fără tick-uri. — consumator
  de verificat

15 Urmăresc un tick prin tot sistemul

administrator

Nou: OpenTelemetry + Tempo + ELK — lecția `devops/tracing-otel-tempo-elk` înainte (de făcut)

- ☐ **F15.1** Un tick se vede ca un singur drum în Tempo. — trace-uri
  de scris
- ☐ **F15.2** Același tick se caută în Kibana după `tickId` sau `traceId`. — loguri
  de scris

16 Cine ești, spus de Keycloak

toți · agentul · administratorul

Nou: Keycloak — lecția `java/spring/spring-security-keycloak` înainte (de făcut)

- ☐ **F16.1** Keycloak emite tokenurile; API-ul devine resource server. — realm în Git
  de scris
- ☐ **F16.2** Agentul își ia tokenul prin client credentials; vechiul `/auth/login` → 404. — —
  de scris

17 O singură poartă: Kong

toți

Nou: Kong + rețele Docker — lecția `devops/kong-gateway` înainte (de făcut)

- ☐ **F17.1** Tot traficul spre aplicații trece prin Kong, cu jwt, rate-limiting și id de corelare. — `kong.yml`
  de scris
- ☐ **F17.2** Aplicațiile nu mai au porturi publicate. — rețele `edge` / `backend`
  de verificat